

Action Learning Project Proposal

A Custom Lightweight Convolutional Neural Network Architecture for Edge-Based Inference using Arduino

Overview:

Nowadays, Deep learning models are achieving great accuracy in specific tasks such as Vision and NLP. Even training these models requires more computation power because of the params. Now imagine deploying them in edge devices so that's where light weight architectures have been introduced like MobileNetV3 and ShuffleNet but they are not always optimized for task specific or edge devices. This project's main motive is to develop a custom lightweight CNN architecture and benchmark its performance, accuracy and implement it in a ML kit named Arduino.

Group Members: DSA – 5

- Revanth Puvaneswaran
- Jatinkumar Parmar
- Narigela Grishmareddy
- Nguyen Ngoc Lam

Goal:

To create a new custom lightweight CNN architecture and implement it in a tiny ML kit named Arduino and analyze its performance in real time.

Objectives:

- Design a lightweight CNN architecture (Classification or Detection) task.
- Optimizing the model for edge inference.
- Benchmark the proposed architecture against available ones.
- Evaluating model with Quantization-Aware Training (QAT) and Post-Training Quantization (PTQ)
- Implementing in ML kit named Arduino

Knowledge and skills required:

- Deep understanding of neural networks especially CNN (Blocks, Kernels etc.)
- Computer Vision
- Edge Computing

Software, tools, system architecture and components anticipated:

- Python (libraries like OpenCV, NumPy)
- ML Frameworks (PyTorch or TensorFlow)
- Deployment Flow
- Arduino

Research expected:

We need to learn the fundamentals of Computer Vision and become a subject matter expert then we research existing available architectures and going to learn how they implemented i.e. layer by layer of networks such as (Batch normalization, pooling, activation functions and many more). Then we will create our own architecture with python as our primary language and ML frameworks, that run on edge devices with ease.

Plan of action:

Week 1:

- Literature Review

Week 2: (Hardcore)

- Becoming a subject matter expert

Week 3 - 4: (Hardcore)

- Setting up the development environment
- Building and evaluating the architecture (Trial & Error) with steps including below
- Developing a custom architecture
- Training the model on publicly available dataset
- Evaluate the baseline performance of the model

Week 5 - 8: (Hardcore)

- Architecture Optimization such as tweaking layers
- Fine-tune the model for better performance
- Implement the architecture in ML kit.

Week 9 - 10:

- Analyze results and compare the custom CNN's performance with existing lightweight models.
- Benchmark the model performance

Week 11 - 12:

- Preparing the final report for the thesis

Dataset Description:

We are going to use some open-source dataset

--> **CIFAR-10**

- The CIFAR-10 dataset consists of 60000 32x32 color images in 10 classes, with 6000 images per class.
- There are 50000 training images and 10000 test images.
- Source: [Click here](#)
- We choose this one because of its small input size as number of trainable parameters will be less which allows us to research effectively rather than worrying about hardware dependencies

Research Papers:

Paper 1: [Click here for link](#)

This paper explains how to build a lightweight CNN architecture from scratch including explainable definitions of every layer etc. Also, the approach integrates two stages of training; dual-input-output model and transfer learning with progressive unfreezing. Which will be more useful for us to find a starting point.

Paper 2: [Click here for link](#)

It explains the same with some modifications in architectures and methods but focuses on specific use case for developing the architecture so we can modify specifically for a use case if need.

ML Model:

We intend to develop a model that runs on any edge devices with ease. Based on the description we were not focusing on the software development part instead our main motive is to focus on research oriented.

After we built our custom CNN and trained it with CIFAR-10 dataset etc. etc. Then we compared our model accuracy with existing SOTA architectures and implemented it in ML kit.

Instead of transfer learning or bulky models we can switch to these compact models for running in edge device smoothly with an acceptable loss in accuracy.

Based on readings with different metrics we will conclude our model performance with existing ones and create a benchmark.